- Semester:- 03
- Subject Code:- 03010803PC03
- Prepared By:- Dhruvil Kumawat(Technical Trainer)
- Parul University

Course Content

Unit :- 01 : Overview of OOP and Java

Programming Paradigms, Procedural vs Object-Oriented Programming, OOP Features, History of Java, Feature of Java, JVM Architecture, JRE, JDK, Java Program Structure, Compilation and Execution Process, Java Environment Setup, JDK Installation, PATH and CLASSPATH, Introduction to IDEs.

Unit :- 02 : Java Language Basics

Variables, Scope of Variables, Primitive Data Types, Reference Data Types, Type Casting, Wrapper Classes, Autoboxing and Unboxing, Arithmetic Operators, Relational Operators, Logical Operators, Assignment Operators, Unary Operators, Bitwise Operators, Operator Precedence, if Statement, if-else Statement, Nested if, switch Statement, for Loop, while Loop, do-while Loop, for - Each Loop, break Statement, continue Statement.

Unit :-03 : Arrays and String

Arrays, One-Dimensional Arrays, Multi-Dimensional Arrays, Array Operations, String Class, StringBuffer, StringBuilder, String Comparison, String Manipulation Methods

Unit :-04 : Object-Oriented Programming (OOP)

Classes and Objects, Constructors, Access Specifiers, Encapsulation, Inheritance, Types of Inheritance, super Keyword, Polymorphism, Method Overloading(Compile time Polymorphism), Method Overriding (Runtime Polymorphism), Abstraction, Abstract Classes, Interfaces, Functional Interfaces, this Keyword, final Keyword, Object Class, and its Methods.

Unit :-05 : Exception Handling

Errors and Exceptions, Checked Exceptions, Unchecked Exceptions, try-catch Block, Multiple catch Blocks, finally Block, throw Keyword, throws Keyword, Custom Exceptions, Exception Handling, Best Practices .

Unit :-06 : Collection Framework in Java

Introduction to Collections Framework, List Interface, Array List, LinkedList, Set Interface, HashSet, Tree Set, Map Interface, HashMap, Tree Map, Iterators, Generics.

Unit :-07 : Advanced Foundations and Java Frameworks

Process Vs Thread, Thread Creation, Thread Life Cycle, Multithreading Concepts, Synchronization, Inter-thread Communication, File Handling, File Class, Byte Streams, Character Streams, Buffered Streams,

Serialization, Deserialization, Lambda Expressions, Stream API, Optional Class.

Unit :-01 :Overview of OOP and Java

Topics List :

- History of Java.
- Programming Paradigms.
- Procedural and Object-Oriented Programming Languages.
- Procedural VS Object-Oriented Programming Language.
- OOP Features.
- Feature of Java.
- Java Internal Architecture
  - JVM Architecture .
  - JDK Architecture.
  - JRE Architecture .
- Java Program Structure.
- Compilation and Execution Process.
- Java Environment Setup.
- JDK Installation.
- PATH and CLASSPATH.
- Introduction to IDEs.

Topic :-01
" History OF Java Programming Language "

Java programming language was originally developed by Sun Microsystems which was initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]). History of even naming of the Java is very interesting. It went under many names.

Founder : James Gosling

Year : 1996

Place: Sun Microsystem Lab .

Team Name: Green Team .



Java Name History :

Green Talk :-

- James Gosling was leading a team named as 'Green' team. Target of this team was to create a new language which can work on multiple electronic devices seamlessly.
- Initially C++ was the original choice to develop the project. James Gosling wanted to enhance C++ to achieve the target but due to high memory usage, that idea was rejected and team started with a new language initially named as Green Talk.
- The file extension used as .gt. Later this language was termed as Oak and finally to Java.

Oak :-

- James Gosling renamed language to Oak. There was an Oak tree in front of his office..
- James Gosling used this name as Oak represent solidarity and Oak tree is the national tree of multiple countries like USA, France, Romania etc..
- But Oak technologies already had Oak as a trademark and James team had to brainstorm another title for the language.

Finally, Java :-

- Team put multiple names like DNA, Silk, Ruby, and Java. Java was finalized by the team. James Gosling tabled Java title based on type of espresso coffee bean.
- Finally, Java was selected as it was quite unique and represented the essence of being dynamic, revolutionary, and fun to say.
- Sun released the first public implementation as Java 1.0 in 1995. It promised **Write Once, Run Anywhere** (WORA), providing no-cost run-times on popular platforms.

S.No	Versions	Date	Description
1.	JDK Beta	1995	Initial Draft version
2.	JDK 1.0	23 Jan 1996	A stable variant JDK 1.0.2 was termed as JDK 1
3.	JDK 1.1	19 Feb 1997	Major features like **JavaBeans**, **RMI**, **JDBC**, **inner classes** were added in this release.
4.	JDK 1.2	8 Dec 1998	**Swing**, **JIT Compiler**, Java Modules, **Collections** were introduced to JAVA and this release was a great success.
5.	JDK 1.3	8 May 2000	Hotspot **JVM**, JNDI, JPDA, Java Sound and support for Synthetic proxy classes were added.
6.	JDK 1.4	6 Feb 2002	Image I/O API to create/read JPEG/PNG image were added. Integrated **XML parser** and XSLT processor (JAXP) and Preferences API were other important updates.
7.	JDK 1.5 or J2SE 5	30 Sep 2004	Various new features were added to the language like foreach, var-args, generics etc.
8.	JAVA SE 6	11 Dec 2006	1. notation was dropped to SE and upgrades done to JAXB 2.0, JSR 269 support and JDBC 4.0 support added.
9.	JAVA SE 7	7 Jul 2011	Support for dynamic languages added to JVM. Another enhancements included string in switch case, compressed 64 bit pointers etc.
10.	JAVA SE 8	18 Mar 2014	Support for functional programming added. **Lambda expressions**, **streams**, **default methods**, new **date-time APIs** introduced.

S.no	Versions	Date	Description
11.	JAVA SE 9	21 Sep 2017	**Module system** introduced which can be applied to JVM platform.
12.	JAVA SE 10	20 Mar 2018	Unicode language-tag extensions added. Root certificates, thread local handshakes, support for heap allocation on alternate memory devices etc were introduced.
13.	JAVA SE 11	5 Sep 2018	Dynamic class-file constants, Epsilon a no-op garbage collector, local-variable support in lambda parameters, Low-overhead heap profiling support added.
14.	JAVA SE 12	19 Mar 2019	Experimental Garbage Collector, Shenandoah: A Low-Pause-Time Garbage Collector, Microbenchmark Suite, JVM Constants API added.
15.	JAVA SE 13	17 Sep 2019	Feature added - Text Blocks (Multiline strings), Enhanced Thread-local handshakes.
16.	JAVA SE 14	17 Mar 2020	Feature added - Records, a new class type for modelling, **Pattern Matching for instanceof**, Intuitive **NullPointerException handling**.
17.	JAVA SE 15	15 Sep 2020	Feature added - **Sealed Classes**, **Hidden Classes**, Foreign Function and Memory API (Incubator).
18.	JAVA SE 16	16 Mar 2021	Feature added as preview - Records, Pattern Matching for switch, Unix Domain Socket Channel (Incubator) etc.
19.	JAVA SE 17	14 Sep 2021	Feature added as finalized - Sealed Classes, Pattern Matching for instanceof, Strong encapsulation of JDK internals by default. New macOS rendering pipeline etc.
20.	JAVA SE 18	22 Mar 2022	Feature added - UTF-8 by Default, Code Snippets in Java API Documentation, Vector API (Third incubator), Foreign Function, Memory API (Second Incubator) etc.

S.no	Versions	Date	Description
21.	JAVA SE 19	20 Sep 2022	Feature added - Record pattern, Vector API (Fourth incubator), Structured Concurrency (Incubator) etc.
22.	JAVA SE 20	21 Mar 2023	Feature added - Scoped Values (Incubator), Record Patterns (Second Preview), Pattern Matching for switch (Fourth Preview),Foreign Function & Memory API (Second Preview) etc.
23.	JAVA SE 21	19 Sep 2023	Feature added - String Templates (Preview), Sequenced Collections, Generational ZGC, Record Patterns, Pattern Matching for switch etc.
24.	JAVA SE 22	19 Mar 2024	Feature added - Region Pinning for G1 garbage collector, foreign functions and memory APIs , multi-file source code programs support, string templates, vector apis (seventh incubator), unnamed variables, patterns, stream gatherers (first preview) etc.
25.	JAVA SE 23	17 Sep 2024	Feature added - Primitive types in patterns, class file APIs, vector APIs (Eighth incubator), stream gatherers (second preview), ZDC, generation mode by default etc.

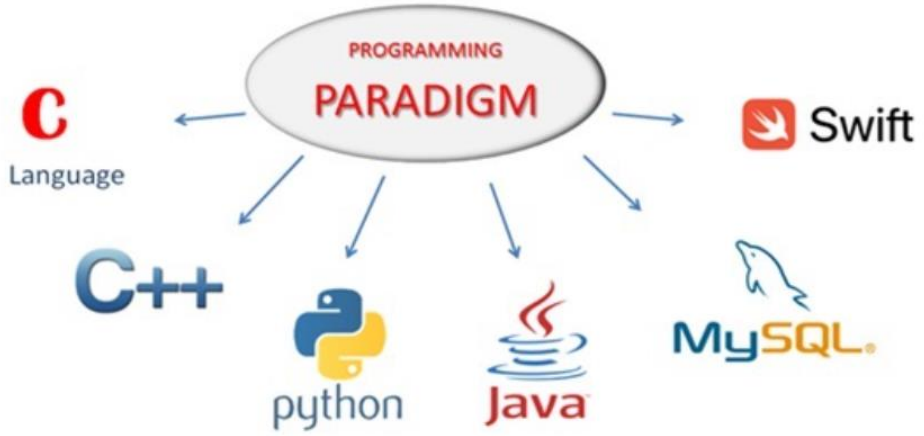
Key Milestones in Java's early History

- 1991 :- Oak Project begins at sun microsystem Lab.
- 1994 :- Pivot to web Browser , Hot Java Browser (Feb 19).
- 1995 :- Java 1.0 Officially released to the Public.
- 1997:- JDK 1.1 introduced inner classes, reflection , JJDC (Feb 19).
- 2010 :- Oracle acquires Sun Microsystems for $7.4 billions.
- 2025 :- Java 24 is the current version (as of March 18, 2025)

Topic :- 02
"Programming Paradigm"

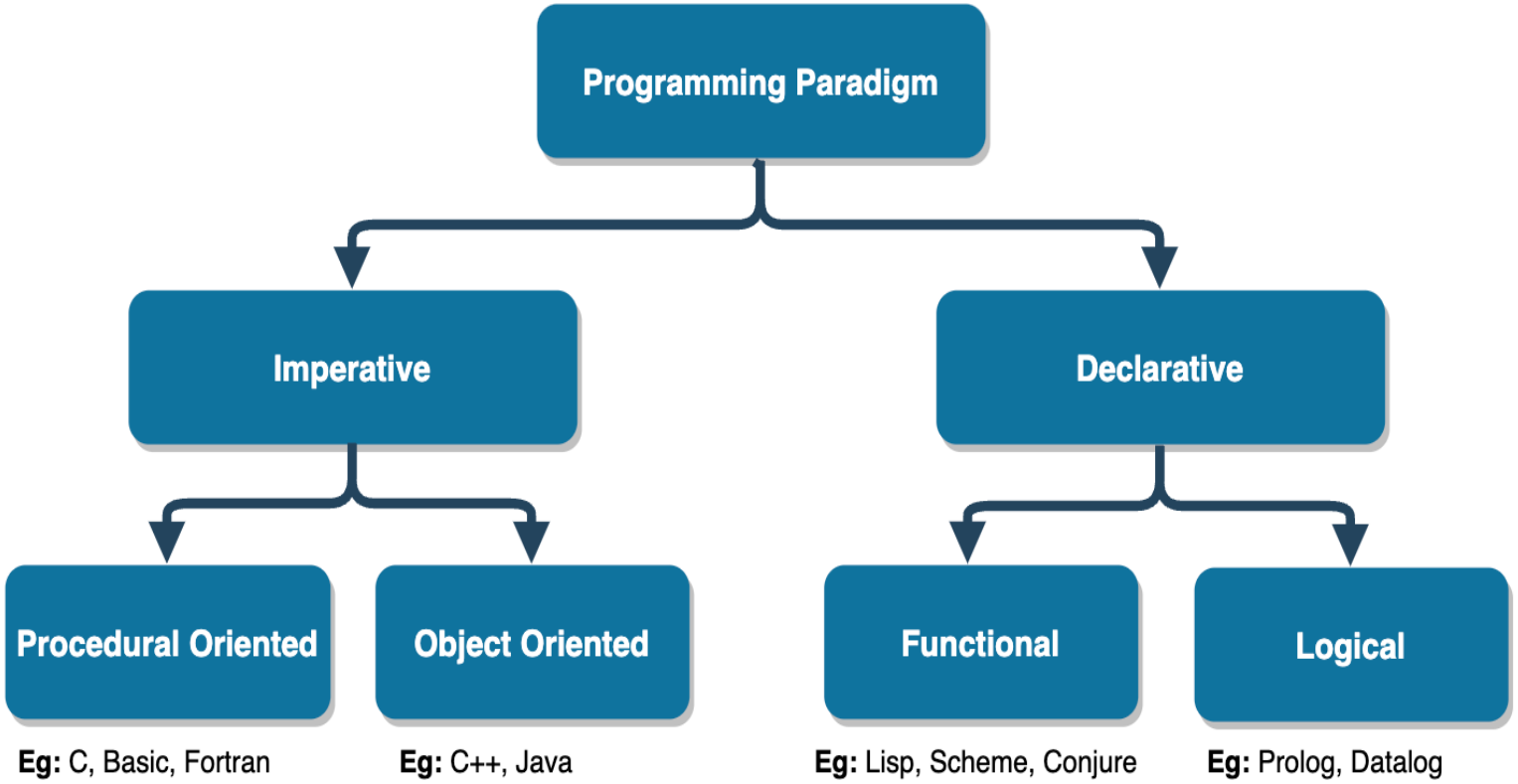
A programming paradigm is a method or style of solving problems using a programming language. It provides a structured approach and set of principles that guide how programs are designed and implemented.

- Different programming paradigms use different techniques and strategies to solve software development problems efficiently.
- A programming language may support one or more paradigms, allowing developers to choose the best approach based on project requirements.



"Types of Programming Paradigm"

Apart from the variety of programming languages available, there are several paradigms that address different demands and challenges in software development. These paradigms are discussed accordingly .



Types of Programming Paradigm :

Imperative Programming Paradigm :

Imperative programming is one of the oldest programming paradigms and is closely related to machine architecture and Von Neumann architecture. It works by changing the program state through assignment statements and executes tasks step-by-step to achieve the desired result.

- Procedural Oriented Paradigm :

This paradigm emphasizes on procedure in terms of under lying machine model. There is no difference in between procedural and imperative approach. It has the ability to reuse the code, and it was boon at that time when it was in use because of its reusability.

Examples of Procedural programming paradigm – C , Fortran , Pascal

- Object-Oriented programming paradigm :

The program is written as a collection of classes and object which are meant for communication. The smallest and basic entity is Object, and all kind of computation is performed on the objects only. More emphasis is on data procedure. It can handle almost all kind of real-life problems which are today in scenario.

Examples of Object-Oriented Programming Paradigm – C++ , Java , Python .

Declarative Programming Paradigm :

Declarative programming is a programming style that focuses on what needs to be done rather than how to do it. It expresses the logic of computation without describing the step-by-step control flow of the program. This paradigm includes logic, functional, and database programming and is often useful for simplifying complex and parallel programming tasks.

- Functional Programming Paradigm :

Functional programming is a paradigm based on mathematical functions where computation is performed through the execution of functions. It focuses on using functions for specific tasks while keeping data loosely coupled and minimizing changes in program state. Some of the languages like Perl, JavaScript mostly uses this paradigm.

Examples of Functional Programming Paradigm – JavaScript , Haskell , Lisp , ML.

- Logical Programming Paradigm :

Logic programming is a paradigm based on facts, rules, and a knowledge base to solve logical problems such as puzzles and reasoning tasks. The program execution works like proving a mathematical statement, where the system derives results from given facts and rules.

Topic :- 03
"Procedural and Object-Oriented Programming Language "

Procedural Programming Language :

- It is a type of programming language that follows a procedure; set of commands or guidelines that must be followed for smooth execution of the program. It works on step-by-step basis.
- It requires the user to tell not only **What to do** but also **How to do it**. Its basic idea is to have a program specify the sequence of steps that implements a particular algorithm.
- It follows a top-down approach. It is carried out in a fixed sequence with a start and end point.

Types of Procedural Programming language :

- Fortran .
- C.
- Cobol .
- Pascal .

Features of Procedural Programming Language -

- Local Variables .
- Global Variables.
- Pre-Defined Functions.
- Parameter Passing.

Object-Oriented Programming :

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around objects rather than functions and logic. An object contains both data (attributes) and methods (behavior), making programs more modular and reusable. OOP helps developers build scalable, maintainable, and real-world applications efficiently.

- Improves code reusability
- OOP is based on the concept of objects and classes
- Makes programs easier to understand and manage

Concepts of Object-Oriented Programming (OOP):

Class :

A class is a blueprint or template used to create objects. It defines the properties (data) and behaviors (methods) that objects will have.

- Contains data members and methods.
- Does not occupy memory until an object is created.

Object :

An object is an instance of a class. It represents a real-world entity and can access the properties and methods defined in the class.

- Occupies memory when created.
- Used to access class members.

Topic :-04
" Procedural vs Object-Oriented Programming Language"

Procedural Programming Language	Object-Oriented Language
In Procedural Programming Language , the program is divided into small parts called functions.	In Object-Oriented Programming , the Program is divided into small parts called Object
Procedural Programming language follows a top-down approach	Object –Oriented Programming language follow a bottom –up approach
There is no access specifiers in Procedural programming	Object-Oriented Programming have access specifies such as Public , private , protected .
Adding new data and function is not easy	Adding New data and function is easy.
In procedural programming overloading is not possible .	Overloading is possible in Object-Oriented programming .
Procedural programming is based on unreal world .	Object-Oriented programming is based on real world .
Examples :- C, Fortran , Pascal , etc.	Examples :- C++ , Java , Python , etc. .

Topic :- 05
"OOP Features"

The Main Features of OOP are :-

Class : A class is a blueprint or template for creating objects. It defines the properties (data) and methods (functions) that objects will have.

Object : An object is an instance of a class. It occupies memory and can access the data and methods of the class.

Encapsulation : Encapsulation means wrapping data and methods into a single unit (class) and restricting direct access to data using access modifiers.

Abstraction : Abstraction means hiding implementation details and showing only essential features to the user.

Inheritance : Inheritance allows one class to acquire the properties and methods of another class.

Polymorphism : Polymorphism means "many forms." A single method can behave differently in different situations.

- Method Overloading (Compile-time Polymorphism)
- Method Overriding (Runtime Polymorphism)

Message Passing : Objects communicate with each other by calling methods.

Topic: - 06
" Features of Java "

The main core features of Java Programming Language includes :

- **Simple Syntax :** Java syntax is very straightforward and very easy to learn. Java removes complex features like pointers and multiple inheritance, which makes it a good choice for beginners.
- **Object-oriented :** Java is a pure object-oriented language. It supports core OOP concepts like, Class , Objects , Inheritance , Encapsulation , Abstraction , Polymorphism .
- **Platform – independent :** Java is **platform-independent** because of **Java Virtual Machine (JVM).**
- When we write Java code, it is first compiled by the compiler and then converted into bytecode (which is platform-independent).
- This byte code can run on any platform which has JVM installed.
- **Scalable :** Java can handle both small and large-scale applications. Java provides features like multithreading and distributed computing that allows developers to manage loads more easily.
- **Portable :** When we write a Java program, the code first get converted into bytecode and this bytecode does not depend on any operating system or any specific computer.

- **Secure and Robust :** Java is a reliable programming language because it can catch mistakes early while writing the code and also keeps checking for errors when the program is running. It also has a feature called exception handling that helps deal with unexpected problems smoothly.
- **Rich Standard Library :** Java provides various pre-built tools and libraries which is known as Java API. Java API is used to cover tasks like file handling, networking, database connectivity (JDBC), security, etc.
- **Support for Mobile and Web Application :** Java offers support for both web and mobile applications.
- For **web development:** Java offers technologies like JSP and Servlets, along with frameworks like Spring and Springboot, which makes it easier to build web applications.
- For **mobile development:** Java is the main language for Android app development. The Android SDK uses special version of Java and its various tools to build mobile apps for Android devices.
- **Functional Programming Features :** Since Java 8, the language has introduced functional programming features such as:
- lambda expression let us to write small block of code in a very easy way without creating full methods.
- Stream API allows data to be processed easily, Instead of writing long loops we can just filter, change, or combine data in a few lines.
- Functional interfaces are interface that contains only one method. They work perfectly with lambda expressions and help us write flexible and reusable code.

Topic :-07
"Java Internal Architecture"

Java Internal Architecture includes :

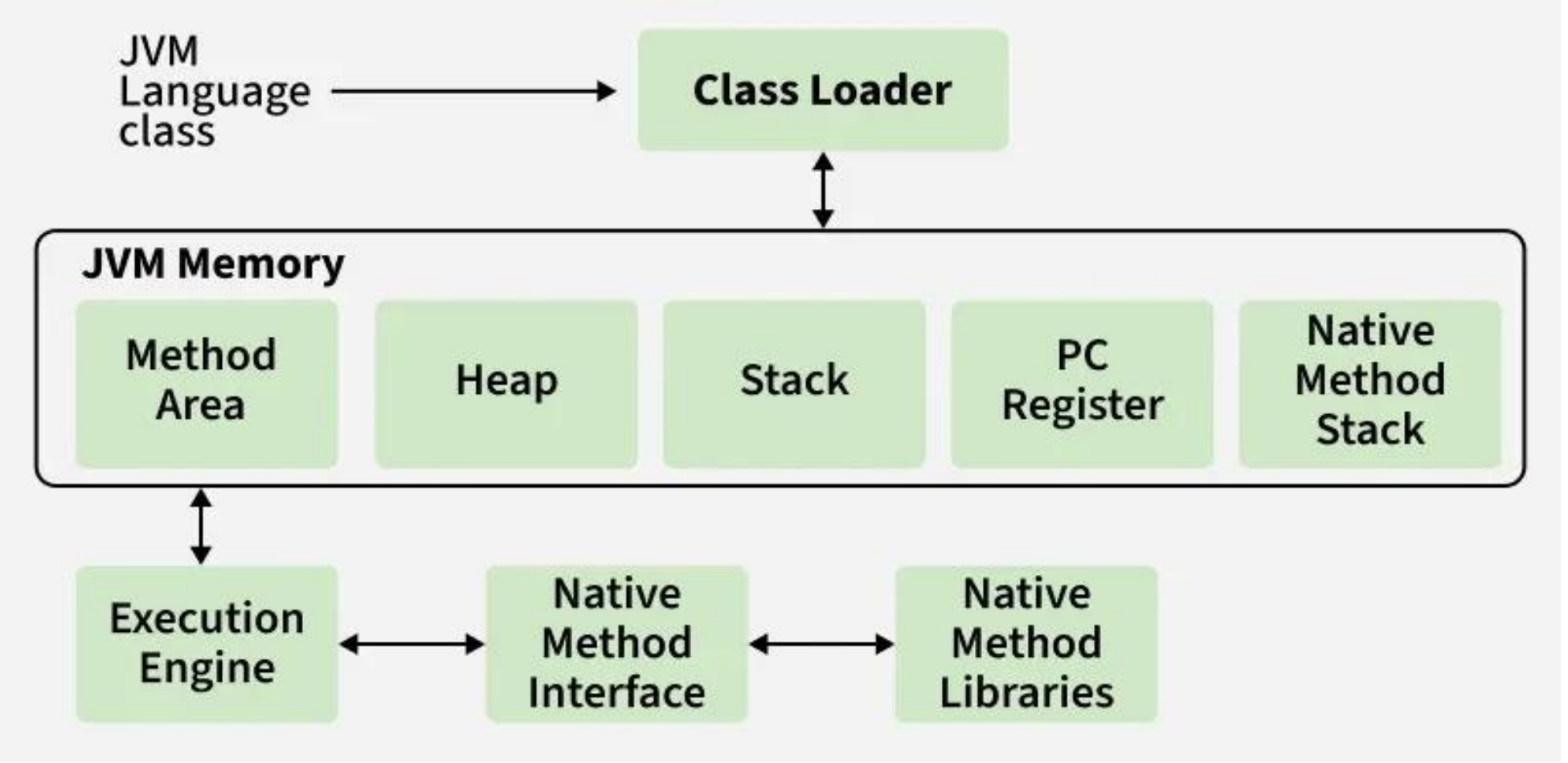
- JVM Architecture.
- JDK Architecture.
- JRE Architecture.

JVM Architecture :

The Java Virtual Machine (JVM) is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platform without modification. JVM acts as an interpreter between Java bytecode and the underlying hardware, providing Java's famous Write Once, Run Anywhere (WORA) capability.

- Java source (.java) -> compiled by javac -> bytecode (.class)
- JVM loads the bytecode, verifies it, links it, and then executes it
- Execution may involve interpreting bytecode or using Just-In-Time (JIT) compilation to convert "hot" code into native machine code for performance
- Garbage collection runs in the background to reclaim memory from unused objects

Architecture of JVM :



Components of JVM Architecture :-

Class Loader Subsystem : It is mainly responsible for three activities.

- **Loading :**
  - Reads .class files and stores class metadata in the Method Area.
  - Creates a Class object in the heap Representing the loaded class.
- **Linking:** Responsible for preparing the loaded class for execution. It includes three steps:
  - **Verification:** Ensures the bytecode follows JVM rules and is safe to execute.
  - **Preparation:** Allocates memory for static variables and assigns default values.
  - **Resolution:** Converts symbolic references into direct references in memory.
- **Initialization:**
  - Assigns actual values to static variables.
  - Executes static blocks defined in the class.

JVM Memory Areas :

- **Method Area:** Stores class-level information like class name, parent class, methods, variables, and static data. Shared across the JVM.
- **Heap Area:** Stores all objects. Shared across the JVM.
- **Stack Area:** Each thread has its own runtime stack; stores method calls, local variables in stack frames. Destroyed when the thread ends.
- **PC Registers:** Hold the address of the currently executing instruction for each thread.
- **Native Method Stacks:** Each thread has a separate stack for native method execution.

Execution Engine :

Execution engine executes the ".class" (bytecode). It reads the byte-code line by line, uses data and information present in various memory area and executes instructions. It can be classified into three parts:

- **Interpreter:** It interprets the bytecode line by line and then executes.
- **Just-In-Time Compiler(JIT):** It is used to increase the efficiency of an interpreter.
- **Garbage Collector:** It destroys un-referenced objects.

Java Native Interface (JNI) :

It is an interface that interacts with the Native Method Libraries and provides the native libraries(C, C++) required for the execution.

Native Method Libraries :

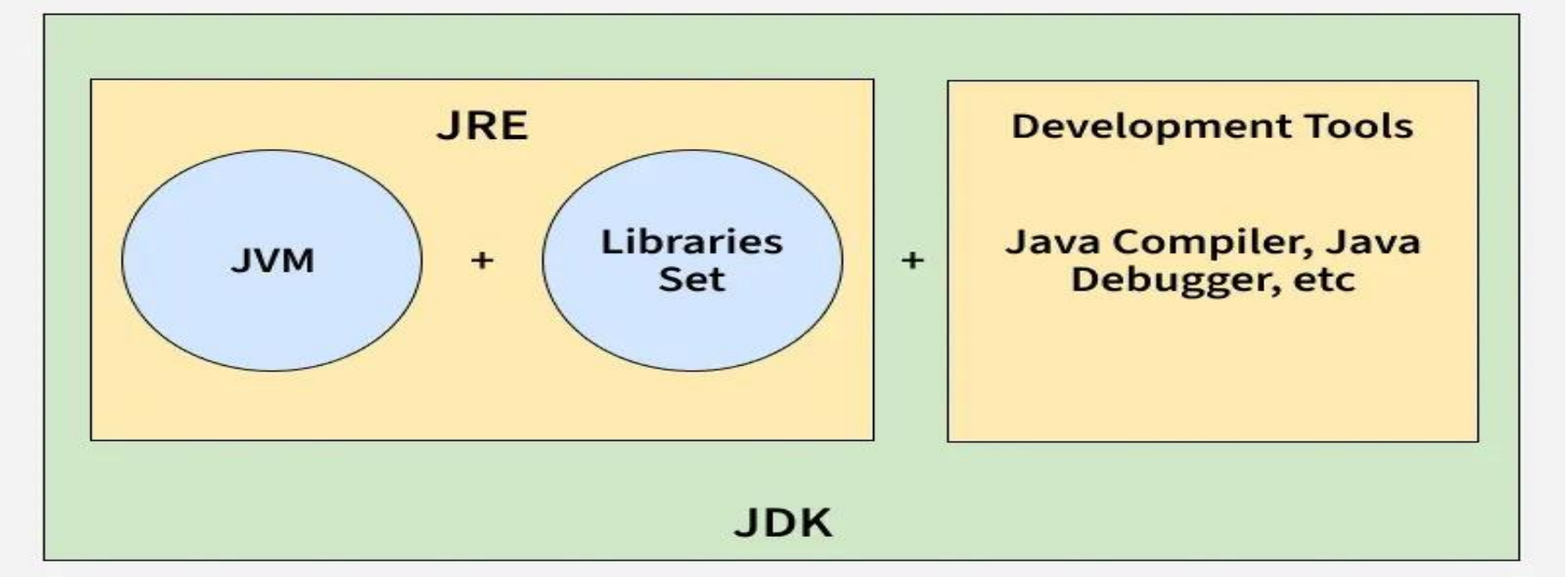
These are collections of native libraries required for executing native methods. They include libraries written in languages like C and C++.

JDK Architecture :

JDK (Java Development Kit) is a software package used to develop and run Java applications. It provides all the necessary tools required for writing, compiling, debugging, and executing Java programs. JDK is mainly used by Java developers to create Java-based applications.

- It includes JRE (Java Runtime Environment) and development tools like the Java compiler (javac).
- It helps developers compile, run, and debug Java programs.
- JDK is platform-dependent and available for Windows, Linux, and macOS.

Architecture of JDK :



Contents of JDK

The JDK has a private Java Virtual Machine (JVM) and a few other resources necessary for the development of a Java Application.

JDK contains :

- Java Runtime Environment (JRE),
- An interpreter/loader (Java),
- A compiler (javac),
- An archiver (jar) and many more.

The Java Runtime Environment in JDK is usually called Private Runtime because it is separated from the regular JRE and has extra content.

Most Popular JDKs ;

- **Oracle JDK:** the most popular JDK and the main distributor of Java11,
- **OpenJDK:** Ready for use: JDK 15, JDK 14, and JMC,
- **Azul Systems Zing:** efficient and low latency JDK for Linux os,
- **Azul Systems:** based Zulu brand for Linux, Windows, Mac OS X,
- **IBM J9 JDK:** for AIX, Linux, Windows, and many other OS,
- **Amazon Corretto:** the newest option with the no-cost build of OpenJDK and long-term support.

JRE Architecture :

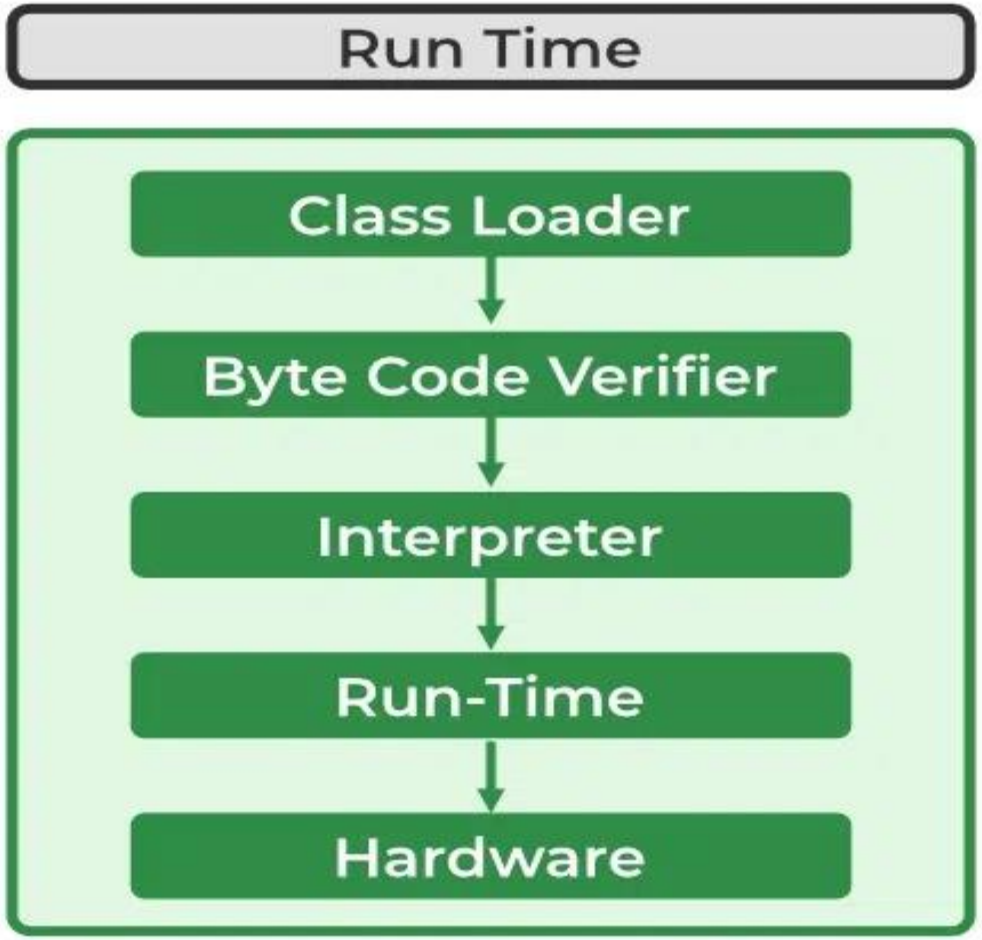
Java Runtime Environment (JRE) is a software package that provides the environment required to run Java applications. It contains the Java Virtual Machine (JVM), core class libraries, and supporting files needed to execute Java bytecode. JRE is a part of the Java Development Kit (JDK) and is responsible for running Java programs on different platforms.

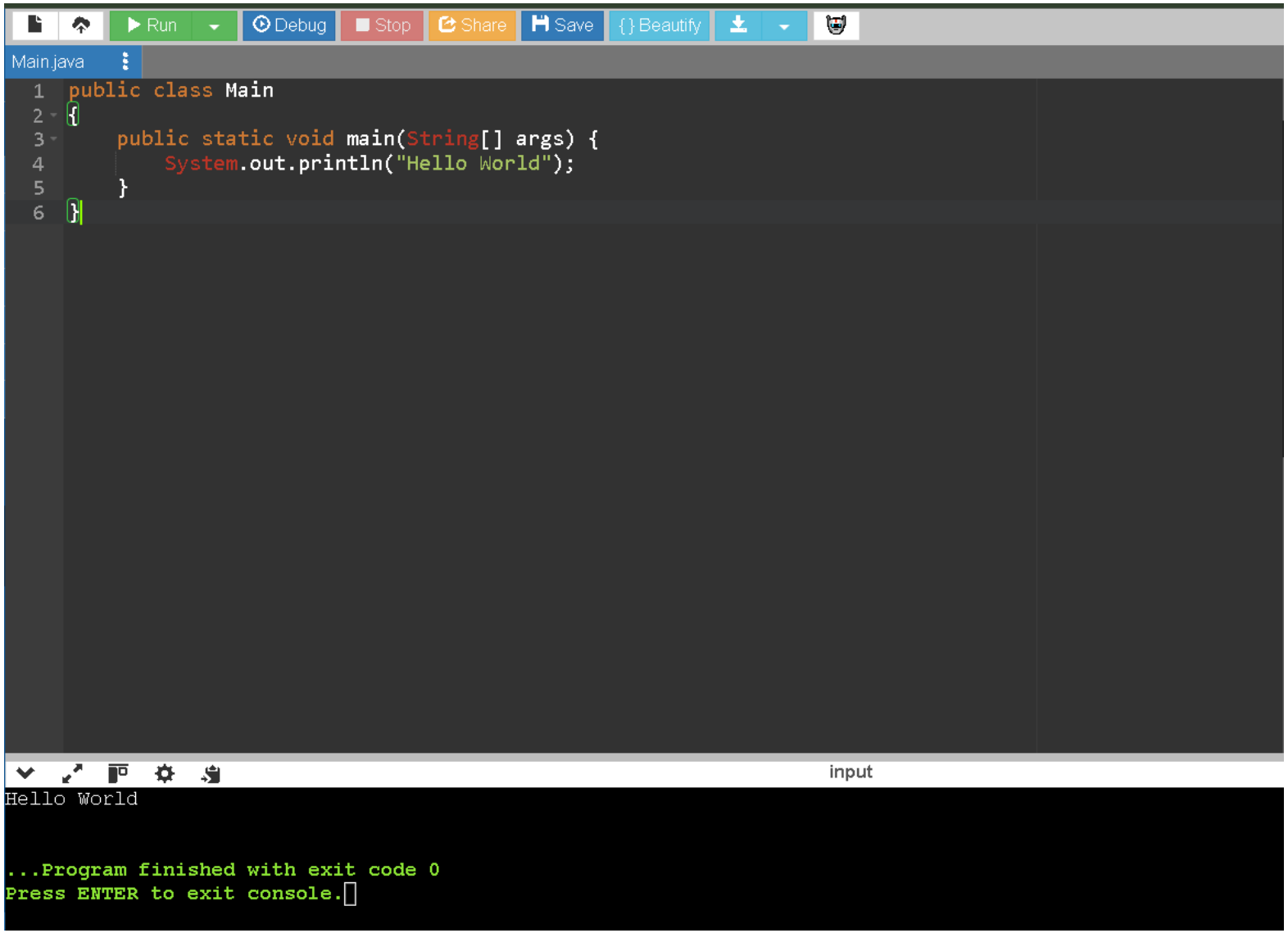
- Executes platform-independent Java bytecode.
- Handles class loading, memory management, and security checks.
- Enables Java's Write Once, Run Anywhere (WORA) capability

Component of JRE :

- Java Virtual Machines.
- Class Libraries.
- Integration Libraries.
- User Interface Libraries.
- Base Libraries.
- Other supporting libraries .

Working Architecture of JRE (Java Runtime Environment) :




Run
Debug
Stop
Share
Save
{} Beautify
Main.java
1 public class Main
2 {
3 public static void main(String[] args) {
4 System.out.println("Hello World");
5 }
6 }
input
Hello World
...Program finished with exit code 0
Press ENTER to exit console.

Topic :-09
"Compilation and Execution Process "

The Java compilation and execution process is unique because it uses a **two-step approach**: it is both **compiled** and **interpreted**. This is what gives Java its famous "Write Once, Run Anywhere" (WORA) capability.

Here is a step-by-step breakdown of how a Java program goes from source code to a running application.

- **Writing the Source Code (`.java`) :**

You start by writing your Java code in a text editor or an IDE (like IntelliJ, Eclipse, or VS Code).

The file **must** be saved with a `.java` extension.

Example: `Main.java`

- **Compilation (`javac`) :**

Once the code is written, it needs to be translated into a language that machines (specifically, the Java Virtual Machine) can understand.

You invoke the Java Compiler using the command: `javac Main.java`

The compiler checks the code for syntax errors.

If the code is error-free, the compiler generates a **`.class` file** containing **Bytecode**

- **Loading and Execution (The JVM) :**

To actually run the program, you use the Java launcher command: java Main. This kicks off the **Java Virtual Machine (JVM)**, which handles the execution through three main phases:

- Phase A: Class Loader

Before execution begins, the JVM's **Class Loader** dynamically loads the .class files into the computer's memory. It also links any other library classes required by your program.

- Phase B: Bytecode Verifier

Before running a single line of code, the JVM puts the bytecode through a strict security check. The **Bytecode Verifier** ensures that the code doesn't violate Java's safety rules (like unauthorized memory access or stack overflows), ensuring malicious code can't easily damage your system.

- Phase C: Execution Engine

Once verified, the **Execution Engine** converts the Bytecode into native machine code (0s and 1s) that your specific computer's operating system and CPU can execute. It does this using two main tools:

- **Interpreter:** Reads and executes the bytecode line-by-line. It's fast to start but can be slow when executing the same code repeatedly.
- **JIT (Just-In-Time) Compiler:** To boost performance, the JIT compiler analyzes the running program and finds "hot spots" (frequently repeated code, like loops). It compiles these parts directly into native machine code so they never need to be interpreted again.

Topic :- 10
"Java Environment Setup "

Setting up your Java development environment is a straightforward process. To write and run Java programs, you need two main components: the **JDK (Java Development Kit)**, which contains the compiler and tools, and an **IDE (Integrated Development Environment)** like IntelliJ IDEA or VS Code to write your code.

Here is the step-by-step guide to getting everything up and running :

Step 1: Download and Install the JDK

The JDK includes the Java Runtime Environment (JRE) and the development tools (like `javac`).

- Go to the official Oracle Java Downloads page or use an open-source alternative like Adoptium Eclipse Temurin.
- Select your Operating System (**Windows**, **macOS**, or **Linux**).
- Download the installer (e.g., the `.msi` or `.exe` installer for Windows, or the `.dmg` installer for macOS).
- Run the downloaded file and follow the installation wizard instructions. Note down the path where Java is installed (usually something like `C:\Program Files\Java\jdk-21` on Windows).

Step 2: Set Up Environment Variables (Mainly for Windows)

Windows needs to know where the Java compiler (`javac`) is located so you can run it from any command prompt window. macOS and Linux installers usually handle this automatically.

For Windows:

- Open the Start Menu, search for **"Edit the system environment variables"**, and open it.
- Click on the **Environment Variables...** button at the bottom right.
- Under **System variables**, click **New...** to create a new variable:
    - **Variable name:** `JAVA_HOME`
    - **Variable value:** *The path to your JDK installation folder* (e.g., `C:\Program Files\Java\jdk-21`)
    - Click **OK**.
- Scroll down in the **System variables** list, find the variable named **Path**, select it, and click **Edit...**.
- Click **New** on the right side and type: `%JAVA_HOME%\bin`
- Click **OK** on all open windows to save the changes.

Step 3: Verify the Installation

To make sure Java is installed and configured correctly, open your terminal or command prompt and run the following verification checks.

1. Check the Java Runtime: java –version

Expected Output: You should see details about the installed Java version (e.g., `java version "21.0.x"`).

2. Check the Java Compiler: javac –version

Expected Output: You should see the compiler version matching your JDK.

Step 4: Install an IDE (Choose One)

While you can write Java in a basic text editor, using an IDE makes coding much faster and easier by catching typos and formatting your code automatically.

- **IntelliJ IDEA (Recommended):** The most popular IDE for Java development. Download the free **Community Edition** from JetBrains.
- **VS Code:** Lightweight and fast. Download VS Code and install the **"Extension Pack for Java"** from the marketplace.
- **Eclipse:** A classic, robust open-source IDE dedicated to Java.

Step 5: Run Your First "Hello World"

Once your IDE is installed, you can test everything with a quick program:

- Create a new Java project in your IDE.
- Create a new file named `Main.java`.
- write the following code:

Code :

```
public class Main {
   public static void main(String[] args) {
      System.out.println("Hello, World! Your Java environment is ready.");
   }
}
```

Click the **Run** button (usually a green play icon next to the code) or right-click the file and select **Run 'Main.main()'**.

Topic:- 11
"JDK Installation "

Here is how to download and install the official JDK step-by-step for **Windows** and **macOS**.

Step 1: Download the JDK Installer

- Visit the official Oracle Java Downloads Page.
- Select your Operating System:
    - **Windows:** Click on the **x64 Installer** link to download the `.exe` file.
    - **macOS:** If you have an Apple Silicon chip (M1/M2/M3), download the **ARM64 DMG Installer**. For older Intel Macs, download the **x64 DMG Installer**.

Step 2: Run the Installation Wizard

For Windows:

- Double-click the downloaded `.exe` file (e.g., `jdk-21_windows-x64_bin.exe`).
- Click **Next** on the welcome screen.
- Note down the installation path (the default is usually `C:\Program Files\Java\jdk-21\`). You will need this in the next step.
- Click **Next**, wait for the installation to finish, and click **Close**.

For macOS:

- Double-click the `.dmg` file to open it.
- Double-click the package icon (the open box) inside the window.
- Follow the prompts, enter your Mac password when requested, and click **Install Software**

Step 3: Set Up Environment Variables (Crucial for Windows)

Windows needs to know exactly where the JDK lives so you can use the `javac` compiler from any folder using the command line.

- Press the **Windows Key**, type **"environment variables"**, and select **Edit the system environment variables**.
- In the System Properties window, click the **Environment Variables...** button at the bottom.
- Under **System Variables** (the bottom section), click **New...**:
    - **Variable name:** `JAVA_HOME`
    - **Variable value:** `C:\Program Files\Java\jdk-21` *(or your specific JDK path)*
    - Click **OK**.
- In that same **System Variables** box, scroll down, select the **Path** variable, and click **Edit...**.
- Click **New** on the right side and type: `%JAVA_HOME%\bin`
- Click **OK** to close all three open windows.

Step 4: Verify the Installation

To make sure everything was installed successfully, you must restart any open terminal windows and run a quick check.

- Open **Command Prompt** (Windows) or **Terminal** (macOS).
- Type the following command to check the compiler: javac –version
- Type the following command to check the runtime environment: java –version

If both commands return the version of java, you just installed (e.g., `javac 21.0.x`), your JDK installation is complete and fully functional!

Topic :- 12
"PATH AND CLASSPATH"

The **Class Path** is a parameter in the Java Virtual Machine(JVM) or the Java compiler that is used by a system or application class Loader to locate and load compiled Java bytecodes stored in the ".class" file.

PATH :

PATH is an environment variable that is used to find and locate binary files like "java" and "javac" and to locate needed executables from the command line or Terminal window. To set the path, we are supposed to include or mention JDK_HOME/bin directory in a PATH environment variable. The PATH cannot be overridden by providing command and PATH is only used by the operation system(OS) to find binary files.

Syntax :

```
// To set PATH in the window OS.
set PATH=%PATH%;C:\Program Files\Java\JDK1.5.10\bin

// To set PATH in Unix OS
export PATH=${PATH}:/opt/Java/JDK1.5.10/bin
```

CLASSPATH :

class path is an environment variable that is used by the application class Loader or system to locate and load the compiled Java bytecodes stored in the .class file. To set CLASSPATH. the CLASSPATH can be overridden by adding class path in the manifest file and by using a command like set - class path. the CLASSPATH is only used by Java Class Loaders to load class files.

Syntax

```
// To set CLASSPATH in window OS.
set CLASSPATH=%CLASSPATH%;C:\Program Files\Java\JDK1.5.10\lib

// To set PATH in Unix OS.
export CLASSPATH=${CLASSPATH}:/opt/Java/JDK1.5.10/lib
```

Difference between PATH and CLASSPATH :

PATH	CLASSPATH
An environment Variable is used by operating system to find the executable files .	An environment Variable is used by java compiler to find the path of classes .
PATH setting up an environment for the operating system . Operating system will look in this PATH for executables .	CLASSPATH Setting up an environment for Java. Java will use to find compiled classes .
Refers to the Operating System .	Refers to the developing environment .
In Path Variable , we must place .\bin folder path.	In class path , we must place .\lib\jar file or directory path in which .java file is available .
PATH is used by CMD Prompt to find binary files.	CLASSPATH is used by the compiler and JVM to find binary files .

Topic :- 13
"Introduction to IDEs"

IDE Stands for Integrated Development Environment Instead of opening four different programs to write, compile, debug, and run your code, an IDE combines everything into a single application. A standard Java IDE includes:

- **The Source Code Editor:** A text editor designed for code. It features **syntax highlighting** (coloring keywords, variables, and strings differently) and **auto-indentation** to keep your code readable.
- **Build Automation Tools:** Instead of manually typing `javac Main.java` and `java Main` every time, an IDE has a simple **"Run" button** (usually a green play icon) that compiles and executes your code instantly.
- **The Debugger:** A tool that lets you pause your program mid-execution line-by-line. This allows you to inspect variables and find exactly where a bug or logic error is happening.
- **Code Completion (IntelliSense):** As you type, the IDE suggests code snippets, methods, and variable names, saving you from memorizing every single Java command.

Types of IDEs :

Desktop IDEs - This type includes the IDEs that can be configured locally. They do not need an active internet connection to build/run programs.

Examples: include Microsoft Visual Studio, Eclipse, NetBeans, etc.

Cloud IDEs - Cloud IDEs eliminate the overhead of configuring the software locally.

Examples: Include Gitpod, AWS Cloud 9, Replit, etc.

Mobile App Development IDEs - These IDEs are specially designed for creating mobile applications.

Examples: Include Android Studio, Flutlab.io, etc.

Database-Specific IDEs - These IDEs (Integrated Development Environments) are specially designed for working with databases.

Examples: Include MySQL Workbench, Oracle SQL Developer, etc.

Common Features of an IDE

Editor.

Compiler.

Debugger.

Build-in Terminal.

Version Control.

Code snippets.

Extensions and Plugins.

The Big Three: Most Popular Java IDEs :-

There are three major IDEs that dominate the Java ecosystem. All of them have free versions and work on Windows, macOS, and Linux.

IntelliJ IDEA (The Modern Standard) : Developed by JetBrains, IntelliJ is currently the most popular and widely used IDE in the professional Java world.

- **Pros:** Incredible code completion, smart refactoring (rewriting code safely), and a highly intuitive interface. Its built-in AI assistant features are top-tier.
- **Cons:** It is resource-heavy and can eat up a lot of RAM.
- **Best For:** Beginners and professionals alike. (The **Community Edition** is completely free).

Eclipse (The Classic) : Eclipse is a legendary open-source IDE that was the industry standard for over a decade.

- **Pros:** 100% free, highly customizable with a massive marketplace of plugins, and very lightweight compared to modern IDEs.
- **Cons:** The user interface looks a bit dated, and managing plugins can sometimes feel clunky.
- **Best For:** Older enterprise projects, academic settings, and computers with lower hardware specs.

NetBeans (The Official Apache IDE) : NetBeans is another open-source IDE, officially supported by the Apache Software Foundation.

- **Pros:** Out-of-the-box support for database connections, great GUI builders (for drag-and-drop desktop apps), and very straightforward to use.
- **Cons:** Lacks the advanced, smart coding features found in IntelliJ.
- **Best For:** Students and developers building classic desktop layout applications.